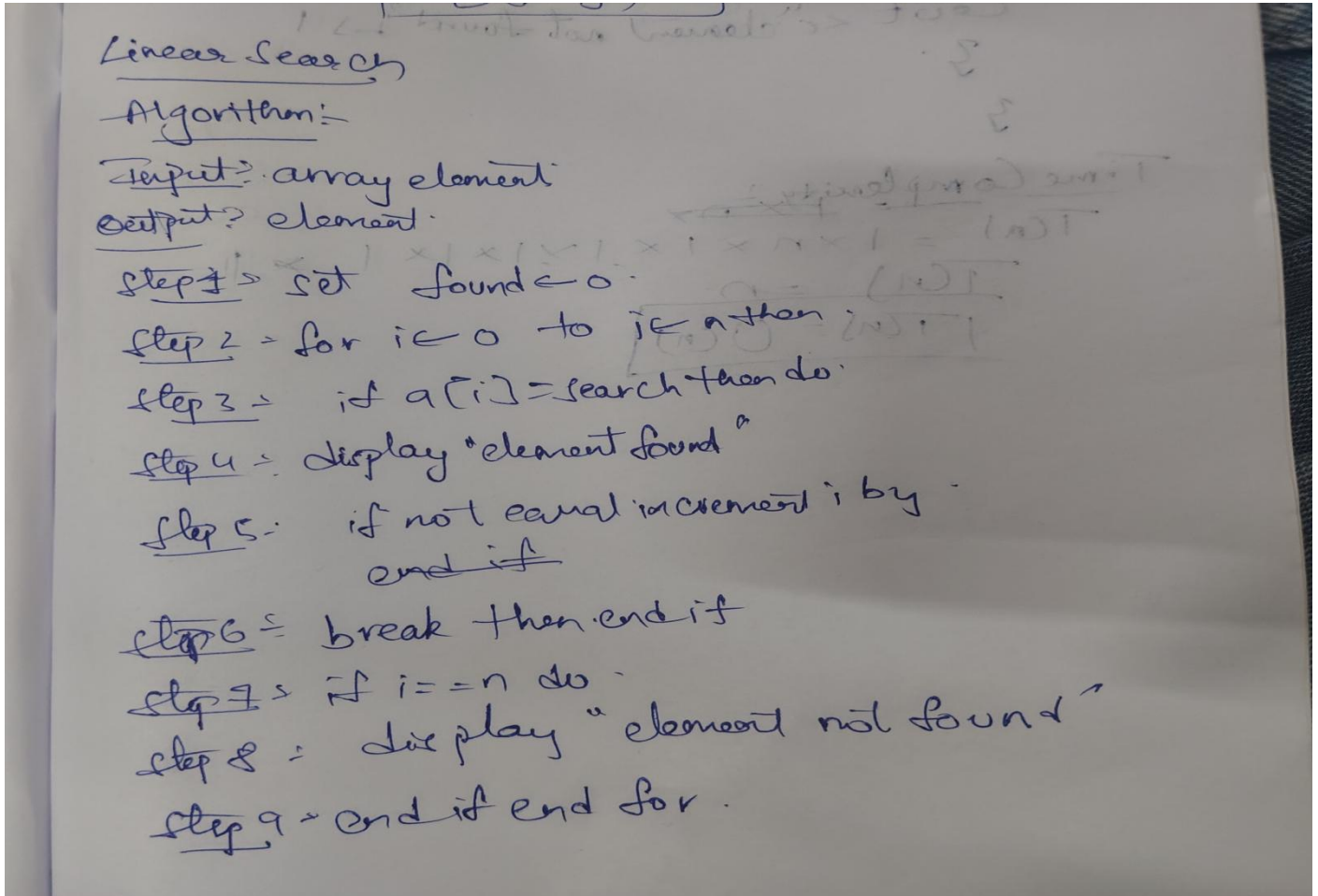


PRACTICAL-2

AIM: Implementation and Time analysis of linear and binary search algorithm.
LINEAR SEARCH

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter the element to search: ";
    cin >> key;

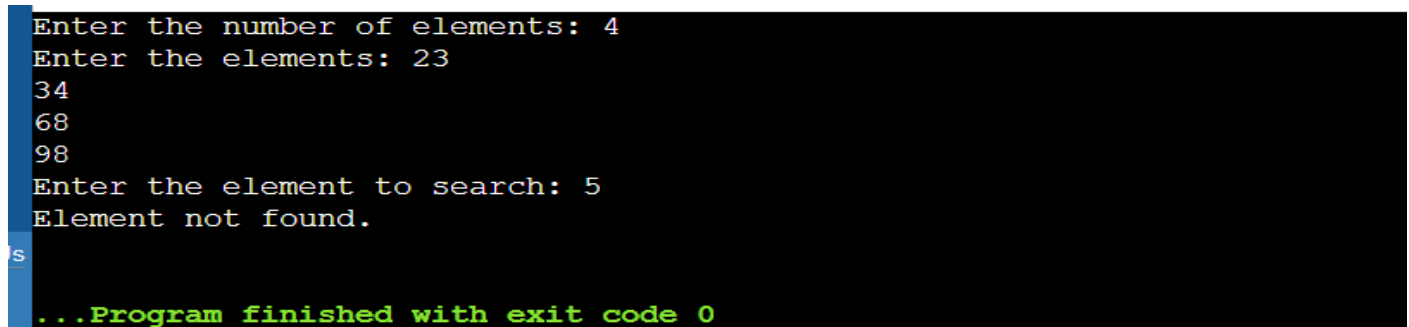
    int found = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            cout << "Element found at position " << i + 1 << endl;
            found = 1;
            break;
        }
    }

    if (found == 0)
        cout << "Element not found." << endl;

    return 0;
}
```

OUTPUT:



```
Enter the number of elements: 4
Enter the elements: 23
34
68
98
Enter the element to search: 5
Element not found.

...Program finished with exit code 0
```

TIME COMPLEXITY:

Time Complexity

```

found = 0;
for (i = 0; i < n; i++) {
    if (arr[i] == search) {
        cout << "element found at " << i;
        found = 1;
        break;
    }
}
if (i == n) {
    cout << "element not found";
}

```

Time Complexity

$$T(n) = 1 \times n \times 1 \times 1 \times 1 \times 1 \times 1 \rightarrow 11$$

$$\frac{T(n)}{1} = n$$

$$T(n) = O(n)$$

1) Best case

Ex: arr = {10, 20, 30, 40, 50}

key = 10. Then is only One-time comparison.

$$T(n) = 1$$

Time complexity = $O(1)$

2) Average case Element found at middle.

Ex: arr = {10, 20, 30, 40, 50}

key = 30 $\rightarrow$ 2 compare

$\frac{n}{2} = 2$ arr[1] > arr[2] = 30

$$T(n) = n/2$$

$$T(n) = O(n)$$

3) Worst case

element found at last

Ex: arr = {10, 20, 30, 40, 50}

key = 50 we check all elements up to n

$$T(n) = n$$

$$T(n) = O(n)$$

BINARY SEARCH

ALGORITHM:

Binary Search :-
Algorithm :-
Input :- sorted elements and key
Output :- position of key
Step 1 :- start
Step 2 :- set low = 0 and high = n - 1
Step 3 :- while low <= high then do
Step 4 :- set mid = (low + high) / 2
Step 5 :- if a[mid] = key then do
Step 6 :- then break
Step 7 :- Else if a[mid] < key then
Step 8 :- set low = mid + 1
Step 9 :- Else high = mid - 1
Step 10 :- if low > high then found
Step 11 :- end

CODE:

```
#include <iostream>
using namespace std;

int main() {
    int n, key;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter the sorted elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    cout << "Enter the element to search: ";
    cin >> key;

    int low = 0, high = n - 1;
    int found = 0;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (arr[mid] == key) {
            cout << "Element found at position " << mid + 1 << endl;
            found = 1;
            break;
        }
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

    if (found == 0)
        cout << "Element not found." << endl;

    return 0;
}
```

OUTPUT:

```
Enter the number of elements: 5
Enter the sorted elements: 45
32
66
77
86
Enter the element to search: 77
Element found at position 4
```

TIME COMPLEXITY:

Time Complexity

low = 0, high = n-1 ; $\rightarrow$ 1
while (low <= high) $\rightarrow$ 1
mid = (low + high) / 2 ; $\rightarrow$ 1
if (a[mid] == key) $\rightarrow$ 1
break ; $\rightarrow$ 1
else if (a[mid] < key) $\rightarrow$ 1
low = mid + 1 ; $\rightarrow$ 1
else high = mid - 1 ; $\rightarrow$ 1

$T(n) = 1 + \log n + \log n + (1 + 1 + \log n + \log n + \log n)$
 $= 2 + 4 \log n$
 $T(n) = O(\log n)$
every iteration divides into half

i) Best case - $\rightarrow$ 1
mid = (low + high) / 2 ; $\rightarrow$ 1
if (a[mid] == key) $\rightarrow$ 1
 $T(n) = 1 + 1 = 2$
 $T(n) = O(1)$

ii) Average case -
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$
no. of divisions = $\log_2 n$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

iii) Worst case -
 $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$

CONCLUSION:

Linear Search and Binary Search were successfully implemented. Linear Search checks elements one by one, so it is simple but slower for large lists. Binary Search divides the list into two parts repeatedly, making it faster, but the list must be sorted. Therefore, ****Binary Search is faster, while Linear Search is simpler and works on unsorted lists.**